

Memory Mapping - Previous Notes

Introduction

A memory map is a combination of shared memory and a file access. It allows us to have one or more files be mapped into the address space of your program. A good question is why you might want to do this? The following are a few notes about memory maps:

- It is an alternative to using shared memory. Just like shared memory, the memory mapped file may be shared among multiple processes. For example, if you want to run multiple instances of a program that share the same information, memory mapping is a great tool.
- When you memory map a file you get back an address that is associated with the file. You are therefore able to treat the file as if it were a structure.
- You can specify protection associated with the memory mapped file. So, you make it so that some programs can only read the data from the files. You can do this with shared memory also.
- There is a limit to how many memory mapped files that you can have. This usually at least 64K, so it will be unusual for this to be an issue..
- You cannot use memory mapping to extend the size of a file.
- The map is not persistent if all accessing programs are terminated. This can be a good thing.
- For more information see: <https://linux.die.net/man/3/mmap>.

Prototype

#include <sys/mman.h>

void *mmap(void *addr, size_t len, int prot, int flags, int fildes, off_t off);

addr - set this to NULL. It is a suggestion to the OS on where to assign the address. Unless you are doing a strange application, it is not relevant.

len - the number of bytes to be mapped. This does not increase the size the memory mapped beyond that of the underlying file,

prot - Specifies the level of protection. The protection is specified by or'ing together the

constants specified below. If the protection specified cannot be supported, the `mmap` call will fail.

`PROT_WRITE` - Data can be written.

`PROT_READ` - Data can be read.

`PROT_EXEC` - Data can be executed. To me this is impressive!

`PROT_NONE` - Data cannot be accessed.

flags - indicates additional requirements. These are or'ed together. There are a lot more.

`MAP_SHARED` - updates to the mapping are visible to other processes mapping the same region and are carried back to the underlying file.

`MAP_PRIVATE` - Changes are private. That is they are not reflected back to the file and not visible to other processes mapping the same file.

`MAP_FIXED` - Interpret *addr* exactly. Don't worry about this one.

fd - the file descriptor for the file to be mapped. Note: this can be closed immediately after this function returns.

off - the offset into the file where we will first access data.

Returns

This function will return the address through which the file may be accessed. It returns `MAP_FAILED` (i.e. `(void *) -1`) if there is an error.

Cleaning up:

When a map is no longer needed, it should be cleaned up. This is done with the `munmap` call. The following is its prototype and a description of its arguments.

```
#include <sys/mman.h>
```

```
int munmap( void *addr, size_t len );
```

This function removes the mapping pages from the `addr` specified to the `len` specified. It returns 0 if successful and -1 if it fails.

The following are a couple examples of using memory mapped files.

[mmap1.cpp](#) - demonstrates setting up a memory map. It sets up the map and records a value in the file.

[mmapTest.cpp](#) - demonstrates initializing mapped memory and displaying it. Although it is good form to unmap the mapped memory I did not do so to show that ,

Write a program to record the contents of a structure into a file and another program to read it. We will have them running at the same time.